Tutorial 4 | Textos e recuperação documental

TF-IDF, similaridade de cosseno e preparação de um RAG · Explicação técnica para o docente

1. Problema de pesquisa: consulta a fontes textuais

O quarto tutorial aborda uma etapa central em aplicações de IA Generativa com documentos: recuperar trechos relevantes antes de solicitar uma resposta textual. A pergunta didática é como localizar definições e explicações em uma pequena coleção documental de Estudos Populacionais, mantendo a identificação de cada fonte.

A coleção é um dicionário Python com quatro documentos sintéticos identificados de DOC-01 a DOC-04. Cada texto contém uma definição breve sobre saldo migratório, envelhecimento populacional, projeções populacionais ou taxa bruta de natalidade. Não são citações de fontes oficiais; foram escritos exclusivamente para a aula. A questão prática é recuperar o documento mais pertinente para uma pergunta escrita em linguagem natural.

2. Bibliotecas e estruturas de dados

- Python nativo (dict, list, strings): armazena pares identificador–texto e preserva uma correspondência entre os textos e suas referências documentais. f-strings constroem a saída legível e o prompt final.
- scikit-learn / feature_extraction.text: TfidfVectorizer converte uma coleção de textos em uma matriz esparsa documento-termo. Cada coluna representa um termo do vocabulário e cada linha, um documento.
- scikit-learn / metrics.pairwise: cosine_similarity calcula a semelhança angular entre o vetor da pergunta e os vetores dos documentos. Quanto maior o valor, mais semelhantes são suas representações TF-IDF.
- NumPy (indiretamente): o array de similaridades devolvido pelo scikit-learn fornece argsort() para obter os índices em ordem crescente; [::-1] inverte a ordem, exibindo primeiro os documentos mais similares.

3. Representação TF-IDF: transformar texto em números

TF-IDF significa Term Frequency–Inverse Document Frequency. O componente TF reflete a presença ou frequência de um termo em um documento. O componente IDF reduz o peso de termos muito comuns no conjunto, favorecendo palavras mais discriminativas. O TfidfVectorizer aplica tokenização, constrói um vocabulário e retorna uma matriz de características textuais, geralmente normalizada.

No notebook, strip_accents="unicode" remove diferenças de acentuação na preparação dos termos, facilitando correspondências como “projeções” e “projecoes”. O mesmo vetorizador que foi ajustado nos documentos transforma a pergunta. Isso é essencial: documentos e pergunta precisam ocupar o

mesmo espaço de características.

```
matriz = vetorizador.fit_transform(textos)
vetor_pergunta = vetorizador.transform([pergunta])
```

O ajuste aos documentos estabelece o vocabulário disponível. Termos que aparecem apenas na pergunta, mas nunca nos documentos, não criam colunas novas durante transform(). O método é lexical: compara padrões de palavras, não compreensão semântica profunda. Parafrasear com sinônimos pode dificultar a recuperação.

4. Similaridade de cosseno e ranqueamento

A similaridade de cosseno mede a orientação entre dois vetores, considerando o produto interno dividido pelo produto das normas. Para vetores TF-IDF com pesos não negativos e normalização usual, os resultados ficam entre zero e um. Um valor alto indica sobreposição ponderada de termos; não representa probabilidade de a resposta estar correta.

```
similaridade(A, B) = (A · B) / (||A|| * ||B||)
```

cosine_similarity(vetor_pergunta, matriz)[0] devolve uma pontuação por documento. argsort()[::-1] fornece o ordenamento. A rotina imprime os dois textos mais próximos, juntamente com identificadores DOC-XX. Esse identificador viabiliza rastreabilidade básica; em sistemas reais, seriam necessários também título, autor, versão, URL, data de coleta, página e trecho original.

5. Recuperação não é geração

O código seleciona o documento melhor posicionado e imprime seu texto. Esse resultado é uma passagem recuperada, não uma resposta redigida por um modelo generativo. Na etapa seguinte, o notebook constrói uma string denominada prompt contendo a pergunta e a fonte selecionada. Ele não chama API, não executa LLM e não completa a etapa de geração de um RAG.

Em uma arquitetura de RAG (Retrieval-Augmented Generation), um sistema busca passagens relevantes, apresenta-as como contexto a um modelo gerativo e solicita uma resposta ancorada nessas passagens. Uma arquitetura completa ainda precisaria de mecanismos de controle de acesso, recuperação em escala, tratamento de documentos, validação de citações, avaliação de fidelidade e tratamento de perguntas sem suporte no acervo.

6. O que a construção do prompt demonstra

O prompt instrui um LLM hipotético a responder usando apenas a fonte, citar o identificador e declarar ausência de evidência quando necessário. Isso é um controle instrucional, não uma garantia: modelos generativos podem interpretar mal as fontes ou produzir afirmações não sustentadas, motivo pelo qual avaliações e verificações externas continuam necessárias.

Quando o código encontra uma pontuação positiva, exibe o texto recuperado. Se nenhuma palavra da pergunta ocorrer no vocabulário indexado de forma compatível, uma pontuação zero pode indicar ausência de correspondência lexical; isso não prova ausência de informação substantiva. Da mesma forma, pontuação positiva não prova que o texto responde à pergunta. Mesmo no exemplo didático, o bloco final de prompt usa o melhor documento sem aplicar um limiar de relevância, uma limitação

importante para comentar em sala.

7. Passo a passo para explicar aos participantes

- Passo 1: identifique documento, conteúdo e código da fonte; mostre por que documentar a procedência é tão importante quanto obter a resposta.
- Passo 2: mostre que texto pode ser representado numericamente sem treinar um modelo generativo. Explique TF-IDF usando a ideia de palavras mais ou menos distintivas.
- Passo 3: formule a pergunta sobre projeções populacionais e observe quais documentos aparecem no topo do ranqueamento.
- Passo 4: destaque o identificador DOC-03 e a diferença entre localizar um trecho e construir uma interpretação sobre ele.
- Passo 5: leia o prompt final e explique onde um LLM entraria em um sistema real, sem afirmar que ele está sendo executado nesta atividade.

8. Desafios e interpretação do resultado

Ao perguntar “O que é saldo migratório?”, o participante deve observar a recuperação do texto tematicamente relacionado. Ao perguntar “Qual era a população brasileira em 2022?”, a coleção não contém o número solicitado. Qualquer resposta numérica seria não sustentada pelo acervo. Essa comparação é útil para abordar abstenção, cobertura documental e limites de fontes.

A proposta também permite discutir erros de busca: um termo comum pode elevar a similaridade sem responder à pergunta; um sinônimo ausente pode reduzir a pontuação de um texto pertinente. Em aplicações reais, alternativas incluem busca lexical BM25, embeddings semânticos, recuperação híbrida, reranqueamento e filtros por metadados, cada qual com custos e limitações.

9. Relevância para pesquisas populacionais e governança

Estudos Populacionais frequentemente dependem de definições operacionais, relatórios metodológicos, legislação, metadados e notas técnicas. Uma infraestrutura de consulta precisa preservar origem, versão e contexto de cada documento. Para materiais sensíveis, são necessários ainda minimização de dados, controle de acesso e respeito à legislação aplicável. Um sistema de busca ou RAG não deve substituir a consulta ao documento original quando a precisão metodológica é determinante.

Perguntas para discussão

- Por que encontrar o documento mais semelhante não garante que ele contenha a resposta?
- Qual a diferença entre uma matriz TF-IDF e um embedding gerado por uma rede neural?
- Qual informação de proveniência faltaria se estivéssemos consultando relatórios reais do IBGE?

Referências para aprofundamento

Salton, G.; Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.

Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

Documentação oficial: https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction e https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

Correspondência com a prática: tutorial de 30 minutos publicado em cbeluzo.github.io/abep-2026/tutoriais/pratica-2h/. Material explicativo; os dados dos exemplos são sintéticos.